

Networking Essentials

Phase 1 · Fundamentals · Estimated time: 2 hours

1. Concept From First Principles

Every network conversation needs an agreed-upon set of rules — a **protocol** — so both sides know how to interpret the bytes flowing between them. This chapter covers the handful of protocols that show up in almost every HLD interview: TCP, UDP, HTTP/HTTPS, and WebSockets.

Both protocols sit at the same layer (the transport layer) directly above IP, and everything above them — HTTP, gRPC, WebSockets, WebRTC — is built as a layer on top of one or the other. That's why this chapter starts here: choosing TCP or UDP is often an implicit choice made by whichever higher-level protocol you pick, so understanding the two underlying trade-offs lets you reason about *why* HTTP behaves the way it does, rather than memorizing HTTP's behavior as a disconnected fact.

TCP vs UDP — the foundational trade-off

TCP (Transmission Control Protocol) is connection-oriented and reliable: before any data flows, client and server perform a three-way handshake (SYN → SYN-ACK → ACK). Every packet is acknowledged, lost packets are retransmitted, and packets are reassembled in the correct order. This reliability costs latency and overhead.

UDP (User Datagram Protocol) is connectionless: the sender fires packets ('datagrams') without waiting for acknowledgment. No handshake, no guaranteed delivery, no guaranteed order — but much lower latency and overhead.

Backend engineer analogy

TCP is a phone call — you dial, wait for the other person to pick up, confirm they can hear you, and if they miss a sentence they say "sorry, what?" and you repeat it. UDP is shouting an announcement across a room — fast, no confirmation anyone heard it, but you don't wait around either. This is exactly why video calls (Careem/Talabat support chat, Zoom) often use UDP-based protocols — a dropped video frame is fine, but waiting for it to be retransmitted (like TCP would) makes the call feel laggy.

HTTP and HTTPS

HTTP is an application-layer protocol built on top of TCP. It defines methods (GET, POST, PUT, DELETE), status codes (200, 404, 500), and a request/response format. HTTPS is HTTP wrapped in TLS encryption — the same request/response model, but the bytes on the wire are encrypted so a network eavesdropper can't read them. Virtually every public API you'll design in an interview should be assumed to run over HTTPS.

A quick but important addition: **HTTP/1.1** opens one connection per request by default (though keep-alive allows reuse), which historically caused "head-of-line blocking" — one slow request could delay others queued behind it on the same connection. **HTTP/2** introduced multiplexing, letting many

requests share a single connection simultaneously without blocking each other, which is part of why HTTP/2 (and gRPC, which is built on it) performs noticeably better under heavy concurrent load. You don't need implementation-level HTTP/2 knowledge for HLD interviews, but knowing it exists explains why modern APIs default to it.

WebSockets

HTTP is fundamentally request-response: the client always speaks first. WebSockets solve the opposite problem — they establish a single persistent TCP connection that both client and server can send messages over, at any time, in either direction (full-duplex). The connection starts as a normal HTTP request that "upgrades" to the WebSocket protocol, then stays open. This is the backbone of chat apps, live order tracking, and live cricket score updates — anywhere the server needs to push data without the client asking first.

2. Real-World Example / Case Study

Careem's live driver-location tracking uses a persistent connection model (WebSockets or similar) so the rider's app receives location updates as they happen, instead of polling an HTTP endpoint every few seconds (which would be both slower and far more wasteful at Careem's scale). Meanwhile, Careem's payment confirmation API is a classic HTTPS request-response call — it doesn't need push updates, just a reliable one-time answer, so plain HTTPS over TCP is the right, simpler choice.

Video/voice features in apps like Talabat's in-app chat-with-rider calls typically rely on UDP-based real-time transport (e.g., WebRTC, which itself often uses UDP) precisely because a slightly glitchy but current audio stream beats a perfectly reliable but delayed one.

A third example worth knowing: Razorpay's webhook system, which notifies merchants when a payment's status changes, is plain HTTPS request-response — Razorpay's server acts as the *client* in that specific call, making a POST request to a URL the merchant registered. This flips the usual mental picture ("my server is always the server") and is a useful reminder that client/server are roles in a given exchange, not fixed identities — the same machine can be a server in one interaction and a client in another, sometimes within the same feature.

3. Diagram

```
TCP handshake (reliable, connection-oriented):
Client --- SYN -----> Server
Client <-- SYN-ACK ----- Server
Client --- ACK -----> Server
[ connection established, data now flows reliably ]

UDP (connectionless, no handshake):
Client --- datagram -----> Server    (fire and forget)
Client --- datagram -----> Server
(no acknowledgment required)

WebSocket (starts as HTTP, then upgrades):
Client --- HTTP GET + Upgrade header --> Server
Client <-- 101 Switching Protocols ----- Server
[ persistent connection: EITHER side can send, anytime ]
```

4. Trade-offs Table

Protocol	Best for	Avoid when
TCP	APIs, file transfer, anything needing correctness	Ultra-low-latency streaming where a dropped packet is acceptable
UDP	Video/voice calls, live game state, DNS queries	Anything requiring guaranteed, ordered delivery (payments!)
HTTP/HTTPS	Standard request-response APIs	True real-time, server-initiated push
WebSockets	Chat, live tracking, live scores/notifications	Simple CRUD APIs — unnecessary complexity and server cost

5. Common Interview Questions

Q1: "Would you use TCP or UDP for a live video streaming feature? Why?"

Model approach: Choose UDP-based transport (or a protocol built on it, like WebRTC/RTP), explain that retransmitting a lost video frame after the fact is worse than simply skipping it, and that low latency matters more than perfect reliability for this use case.

Q2: "How would you design real-time order status updates without WebSockets?"

Model approach: Describe short-interval polling or long polling as fallbacks, explain their cost (wasted requests, added server load, higher latency for the update to reach the client) versus WebSockets, and note that many production systems use polling as a pragmatic first version before investing in WebSocket infrastructure.

Q3: "A webhook receiver is occasionally getting duplicate delivery of the same event — why, and how do you handle it?"

Model approach: Explain that most webhook senders use "at-least-once" delivery over HTTP — if they don't receive a timely 200 OK (perhaps due to a network blip on either side), they retry, which can cause the same event to arrive twice. The fix is making the receiver idempotent: track processed event IDs and skip ones already handled, a concept Phase 2, Chapter 9 covers in full.

6. Practice Exercises

- 1 For each of these features, decide TCP or UDP and justify it in one sentence: (a) uploading a profile photo, (b) a multiplayer game's player-position updates, (c) placing a food order.
- 2 Draw the WebSocket upgrade handshake from memory.
- 3 Explain why HTTPS doesn't remove the need to think about TCP — i.e., what layer does encryption sit on top of?

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"TCP vs UDP" — PowerCert Animated Videos	Clean animated breakdown of the core difference.
English	"WebSockets: Real-Time Communication Explained" (System Design for Beginners)	Frames WebSockets specifically for system design interviews.
Hindi	"TCP vs UDP Explained in Hindi Difference Between TCP and UDP"	Direct Hindi comparison with examples.
Hindi	"Lec-77: TCP vs UDP differences in Hindi" — Gate Smashers	Popular Hindi CS-fundamentals channel, detailed and structured.

8. Key Takeaways

- TCP trades speed for reliability: handshake, acknowledgments, ordered delivery, retransmission.
- UDP trades reliability for speed: no handshake, no guarantees, minimal overhead.
- Choose UDP-based transport for real-time audio/video where a dropped packet is acceptable.
- HTTP/HTTPS is a request-response protocol built on TCP; the client always initiates.
- WebSockets upgrade an HTTP connection into a persistent, full-duplex channel for true real-time features.
- Most systems use a mix: HTTPS for standard APIs, WebSockets or UDP-based protocols only where real-time genuinely matters.